

Assignment 4

Aisha Abdisamad (101297747)

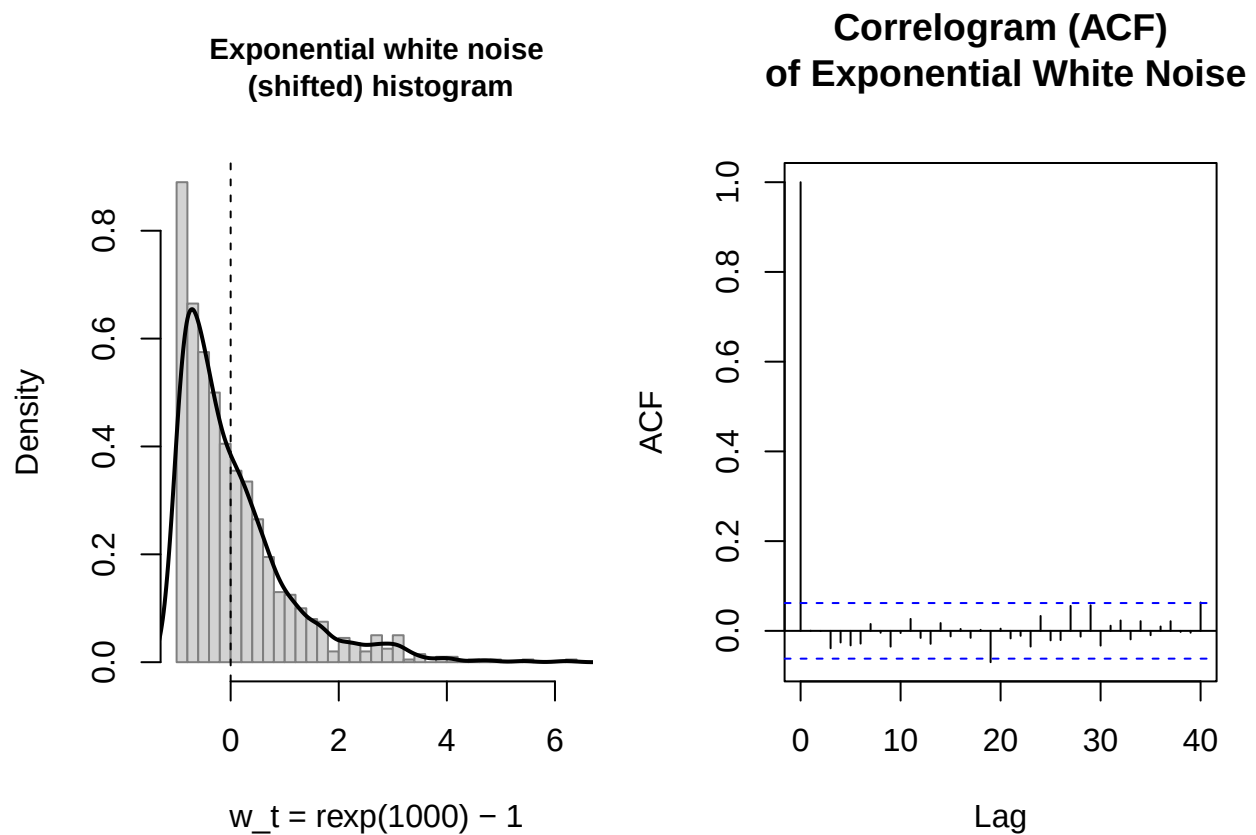
2025-08-14

Question 1: Simulate discrete white noise from an exponential distribution and plot the histogram and the correlogram. For example, you can use the R command `w <- rexp(1000) - 1` for exponential white noise. Comment on the plots.

```
# Simulate exponential white noise
set.seed(123)
w <- rexp(1000) - 1
head(w)
```

```
## [1] -0.1565427 -0.4233897  0.3290549 -0.9684226 -0.9437890 -0.6834988
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 5, 1)) # more top margin
# Histogram
hist(w,
      breaks = 30,
      main = "Exponential white noise\n (shifted) histogram",
      xlab = "w_t = rexp(1000) - 1",
      freq = FALSE,
      border = "grey50",
      cex.main = 0.95)
lines(density(w), lwd = 2)
abline(v = 0, lty = 2)
# Correlogram
acf(w,
     main = "Correlogram (ACF)\n of Exponential White Noise",
     lag.max = 40,
     cex.main = 0.95)
```



```
par(mfrow = c(1, 1))
```

- The histogram shows the expected right-skewed shape of an exponential distribution, even after shifting by 1 to roughly center the values around zero. Most points are close to zero or slightly negative, with a long tail extending to the right. This confirms the data is skewed and not normally distributed.
- The correlogram (ACF) shows auto correlations at nonzero lags are very close to zero and fall within the confidence bounds, with no visible patterns. This supports that the data behaves like white noise which is random and independent.

Question 2:

(a) Simulate time series of length 100 from an AR(1) model with alpha equal to -0.9, -0.5, 0.5, and 0.9. Estimate the parameter of each model and make predictions for 1 to 10 steps ahead.

```
set.seed(123)
n <- 100
alphas <- c(-0.9, -0.5, 0.5, 0.9)

# simulate
x_list <- lapply(alphas, function(a) arima.sim(model = list(ar = a), n = n))
```

```

# fit AR(1) with forecast::Arima
library(forecast)

## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo

fit_list <- lapply(x_list, function(x) Arima(x, order = c(1,0,0)))

# estimated ar1
est <- sapply(fit_list, function(f) coef(f)["ar1"])
round(est, 3)

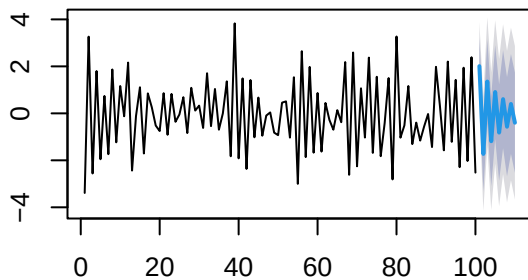
##      ar1      ar1      ar1      ar1
## -0.825 -0.386  0.429  0.861

# 10-step forecasts
fc_list <- lapply(fit_list, function(f) forecast(f, h = 10))

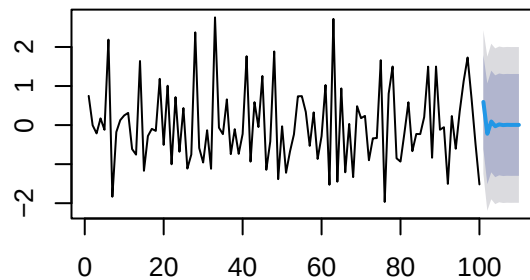
# quick forecast plots, 4-up
op <- par(mfrow = c(2,2), mar = c(4,4,3,1))
for (i in seq_along(alphas)) {
  plot(fc_list[[i]], main = paste("AR(1), alpha =", alphas[i]))
}

```

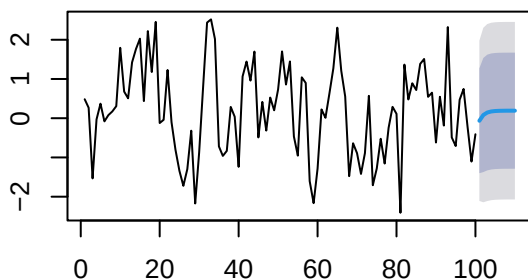
AR(1), alpha = -0.9



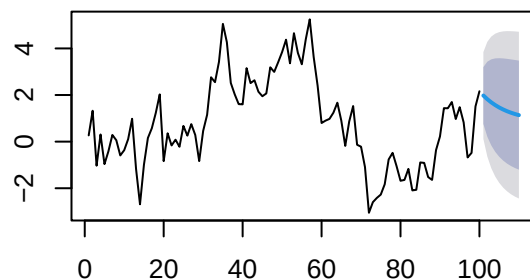
AR(1), alpha = -0.5



AR(1), alpha = 0.5



AR(1), alpha = 0.9



```
par(op)
```

(b) Simulate time series of length 100 from an AR(1) model with alpha equal to 1.01, 1.02, and 1.05. Estimate the parameters of these models.

```
# Simulate time series for (b)
set.seed(321)
n <- 100
alphas_b <- c(1.01, 1.02, 1.05)

sim_ar1_ns <- function(a, n) {
  w <- rnorm(n)
  x <- numeric(n); x[1] <- w[1]
  for (t in 2:n) x[t] <- a * x[t-1] + w[t]
  ts(x)
}
x_list_b <- lapply(alphas_b, sim_ar1_ns, n = n)
est_alpha <- function(x) {
  xt <- x[2:length(x)]
  xt_1 <- x[1:(length(x)-1)]
  unname(coef(lm(xt ~ 0 + xt_1))[[1]])
}
alpha_hat_b <- sapply(x_list_b, est_alpha)
data.frame(alpha_true = alphas_b, alpha_est = round(alpha_hat_b, 3))
```

```
##   alpha_true alpha_est
## 1      1.01      0.990
## 2      1.02      1.005
## 3      1.05      1.050
```

Question 3:

(a) Show that the series in the assignment pdf is non-stationary.

```
# Show non-stationarity
phi_coef <- c(1, -1.5, 0.5)
phi <- c(1, -1.5, 0.5)
rts <- polyroot(phi); rts
```

```
## [1] 1+3.56945e-20i 2-3.56945e-20i
```

```
Mod(rts)
```

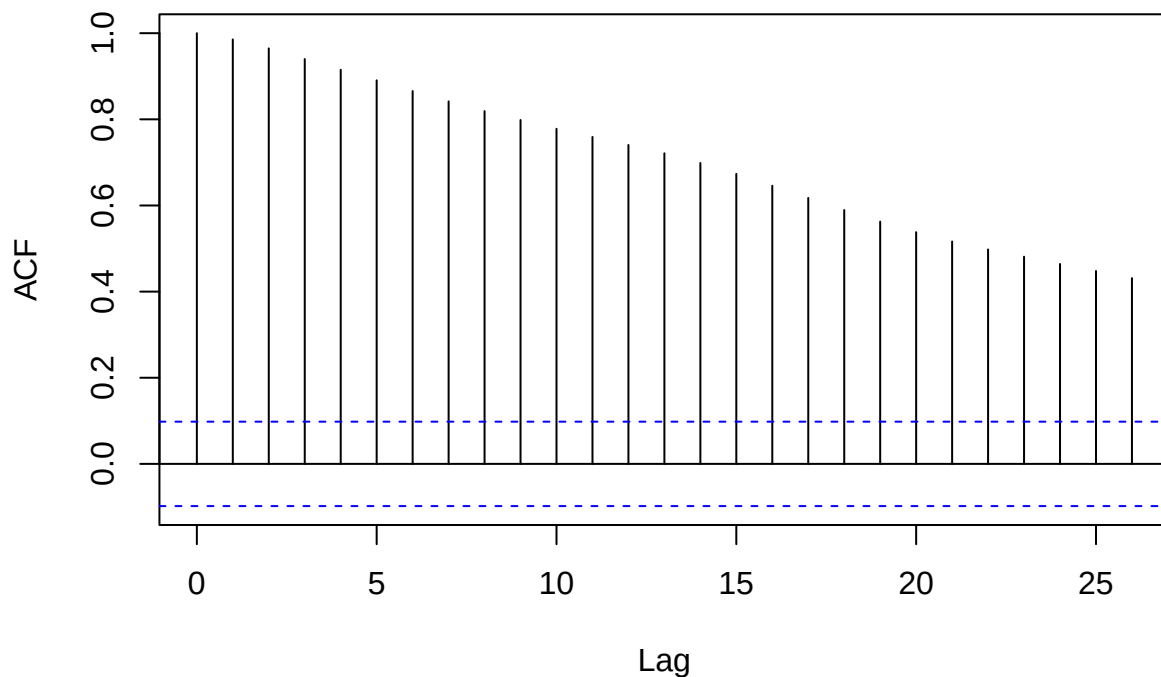
```
## [1] 1 2
```

```

set.seed(1)
n <- 400
w <- rnorm(n)
x <- numeric(n); x[1:2] <- 0
for (t in 3:n) x[t] <- 1.5*x[t-1] - 0.5*x[t-2] + w[t]
acf(x, main = "ACF of x_t (slow decay means non-stationary)")

```

ACF of x_t (slow decay means non-stationary)



- The ACF of the simulated series decays very slowly rather than dropping quickly toward zero. In a stationary AR process, correlations usually die out rapidly, but here they persist across many lags. This pattern confirms that the process is non-stationary.

(b) Write down the model for YT Show that YT is stationary.

```

# root check for y_t's AR polynomial (1 - 0.5 B)
roots_y <- polyroot(c(1, -0.5))
Mod(roots_y)

```

```
## [1] 2
```

```

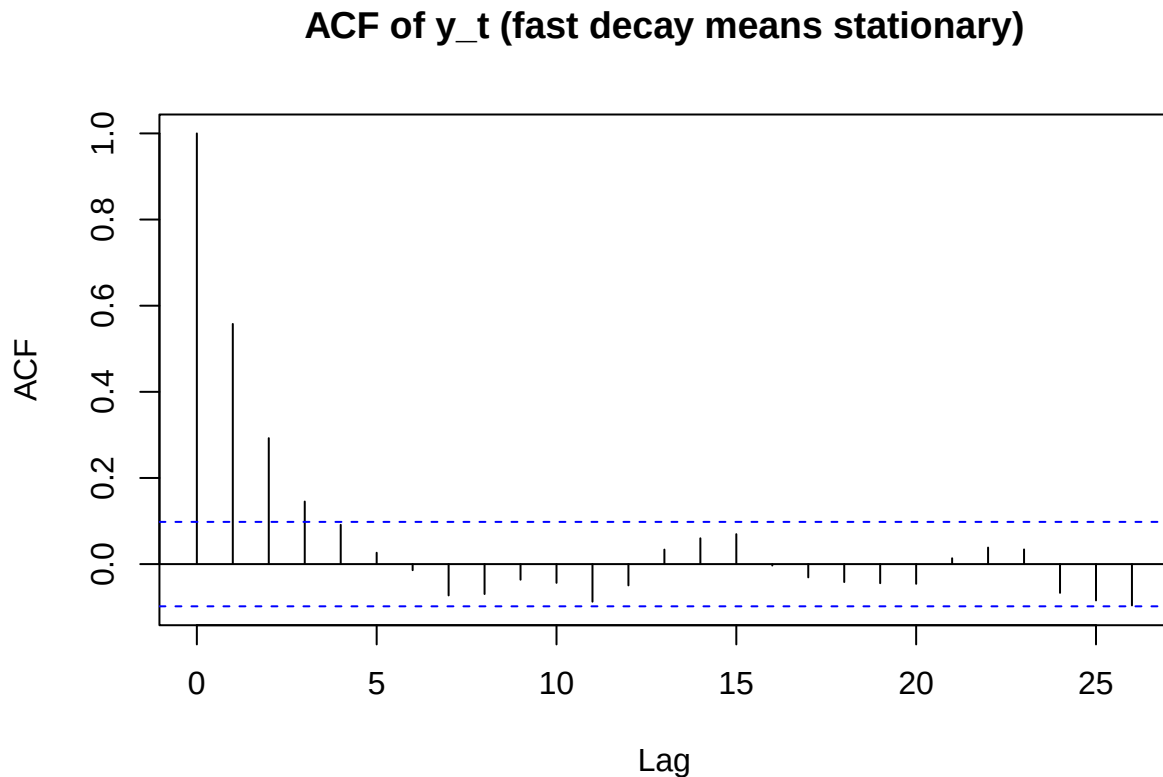
# simulation & fit
set.seed(7)
y <- arima.sim(list(ar = 0.5), n = 400)

```

```
fit_y <- arima(y, order = c(1,0,0), include.mean = TRUE)
round(coef(fit_y)["ar1"], 3)
```

```
## ar1
## 0.559
```

```
acf(y, main = "ACF of y_t (fast decay means stationary)")
```



- The ACF of the simulated series decays very fast, dropping quickly toward zero. In a stationary AR process, correlations usually die out rapidly. This pattern confirms that the process is stationary.

(c) Simulate a series of 1000 values for X_T , placing the simulated data in X and use the simulated values to produce a series of 999 values of Y_T placing this series in the vector Y .

```
# Simulate a series of 1000 values for XT
set.seed(1)
n <- 1000
w <- rnorm(n)
x <- numeric(n); x[1:2] <- 0
for (t in 3:n) x[t] <- 1.5 * x[t-1] - 0.5 * x[t-2] + w[t]
y <- diff(x) # length 999
length(x); length(y)
```

```
## [1] 1000
```

```
## [1] 999
```

(d) Fit an AR model to Y . Give the fitted model parameter estimates and a 95% confidence interval for the underlying model parameters based on these estimates. Compare the confidence intervals to the parameters used to simulate the data and explain the results.

```
# Fit AR model to Y
fit_y <- arima(y, order = c(1,0,0), include.mean = TRUE)
# Parameter estimates
round(fit_y$coef, 3)
```

```
##          ar1 intercept
##      0.445      -0.022
```

```
# 95% confidence interval for AR parameter
round(confint(fit_y), 3)
```

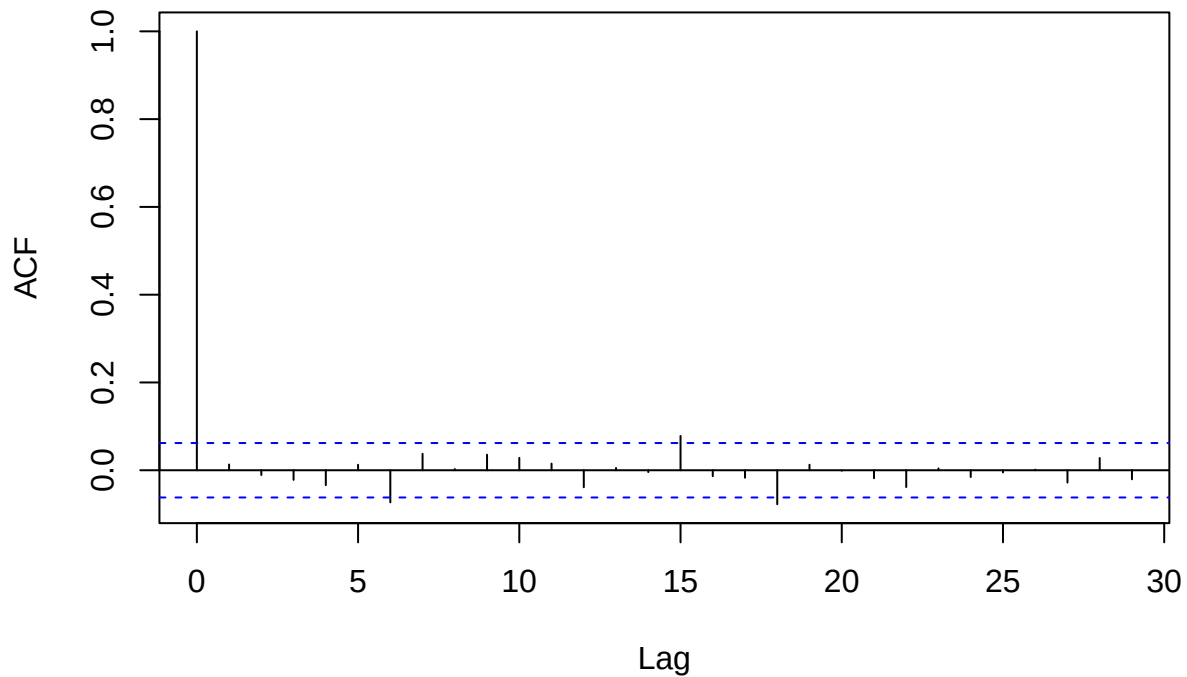
```
##          2.5 % 97.5 %
## ar1          0.389  0.500
## intercept -0.137  0.093
```

- The AR(1) estimate came out to about 0.445, which is close to the true value of 0.5 that we used in the simulation. The 95% confidence interval for the AR parameter is (0.389, 0.500), and since 0.5 is inside this range, the model is doing a good job at capturing the true parameter. The intercept is small (-0.022) and its interval (-0.137, 0.093) includes zero, which makes sense because the true model didn't have an intercept. Overall, the fitted model lines up well with the way we generated the data.

(e) Plot the correlogram of the residuals of the fitted model and comment.

```
# Correlogram of residuals from AR(1) fit
resid_y <- residuals(fit_y)
acf(resid_y, main = "ACF of residuals from AR(1) fit")
```

ACF of residuals from AR(1) fit

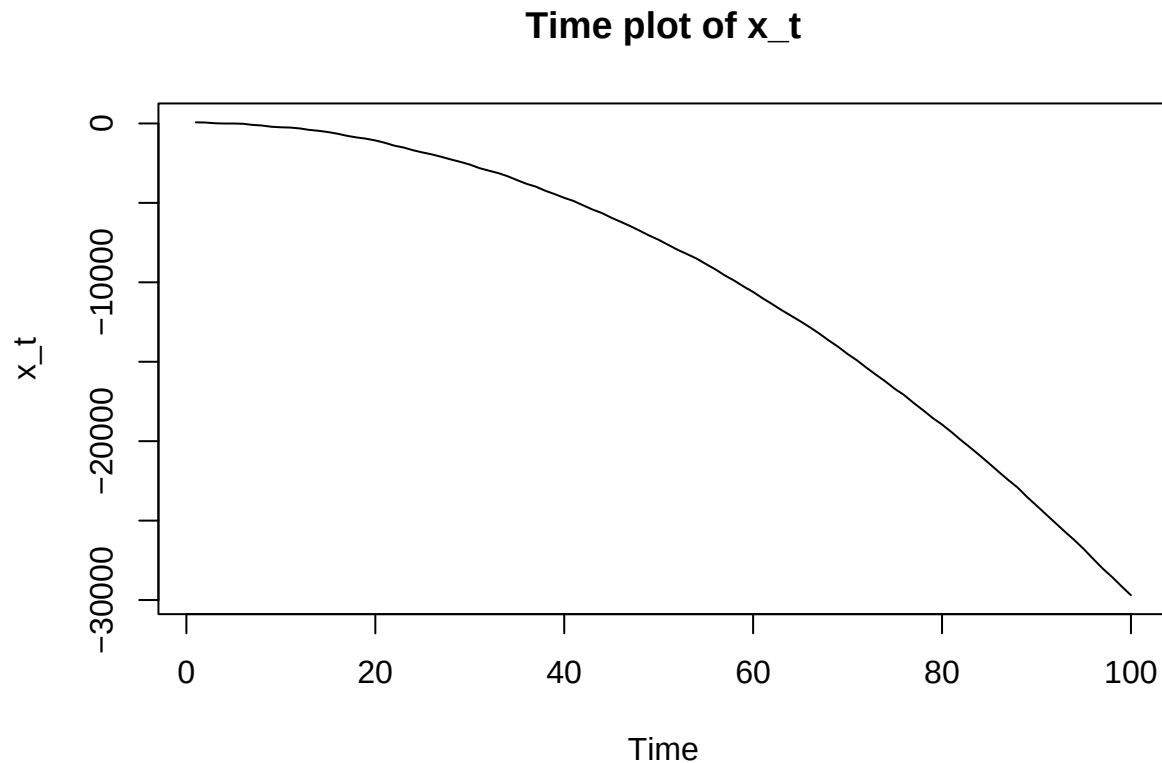


- The ACF of the residuals shows no big spikes outside the blue bounds, which means the AR(1) model has done a good job capturing the dependence in the data. The residuals look like white noise now, so the model seems appropriate.

Question 4:

(a) Produce a time plot for the equation listed in question 4 a

```
set.seed(10)
n <- 100
t <- 1:n
# AR(1) noise
w <- rnorm(n, sd = 25)
z <- numeric(n)
z[1] <- w[1]
for (i in 2:n) z[i] <- 0.5 * z[i-1] + w[i]
# Construct series and plot
x <- 70 + 2*t - 3*t^2 + z
plot.ts(x, main = "Time plot of x_t", ylab = "x_t")
```

(b) Fit a quadratic trend to the series $\{X_T\}$. Give the coefficients of the fitted model

```
# Quadratic trend fit for  $x_t$ 
t <- 1:n
fit <- lm(x ~ t + I(t^2))
summary(fit)$coefficients
```

```
##              Estimate Std. Error    t value    Pr(>|t|)
## (Intercept)  53.904130  8.73014141    6.174485  1.547134e-08
## t            1.998477  0.39899119    5.008825  2.455761e-06
## I(t^2)       -2.997332  0.00382737  -783.130869  3.636095e-186
```

- The coefficients for the model are as follows:
- Intercept: 53.904
- Linear Term: 1.998
- Quadratic term: -2.997

(c) Find a 95% confidence interval for the parameters of the quadratic model and comment.

```
# 95% Confidence interval for quadratic model parameters
ci <- confint(fit, level = 0.95)
lm(x ~ t + I(t^2))
```

```
##
## Call:
## lm(formula = x ~ t + I(t^2))
##
## Coefficients:
## (Intercept)          t          I(t^2)
##      53.904       1.998       -2.997
```

```
round(ci, 3)
```

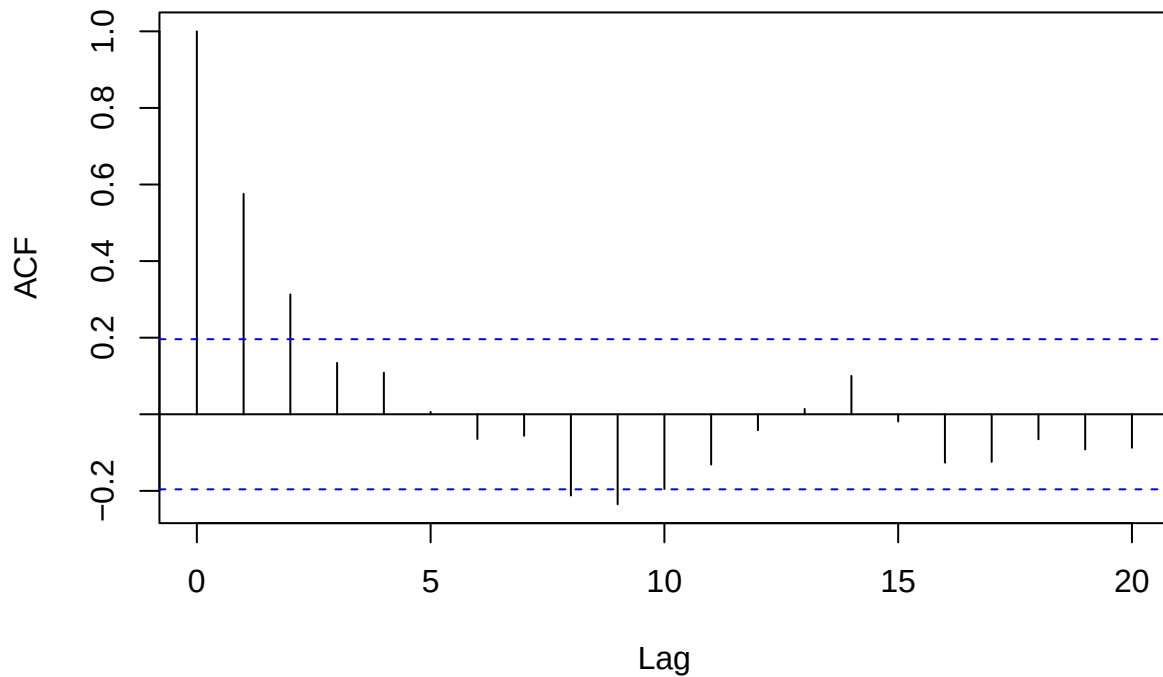
```
##              2.5 % 97.5 %
## (Intercept) 36.577 71.231
## t           1.207  2.790
## I(t^2)      -3.005 -2.990
```

- All three confidence intervals don't include zero, so the intercept, linear, and quadratic terms are clearly significant. The intervals are super tight, which shows the estimates are precise. The only catch is the errors follow an AR(1) process, so the standard errors might be a bit underestimated, meaning we'll need to adjust for that later.

(d) Plot the correlogram of the residuals and comment.

```
# Correlogram of residuals
resid_fit <- residuals(fit)
acf(resid_fit, main = "ACF of residuals from quadratic trend")
```

ACF of residuals from quadratic trend



- The correlogram shows strong positive autocorrelation at the first few lags, and the ACF decays slowly instead of cutting off quickly. This means the residuals are not white noise but there's still dependence left in the data. That makes sense because the errors were generated from an AR(1) process, so we'll need to specifically model that structure later.

(e) Refit the model using GLS. Give the standard errors of the parameter estimates and comment.

```
# GLS refit with AR(1) errors
library(nlme)
```

```
##
## Attaching package: 'nlme'
```

```
## The following object is masked from 'package:forecast':
##
##      getResponse
```

```
gls_fit <- gls(x ~ t + I(t^2), correlation = corAR1(form = ~ t))
summary(gls_fit)$tTable
```

```
##           Value      Std.Error    t-value      p-value
```

```
## (Intercept) 57.243498 17.584870363    3.255270  1.560583e-03
## t           1.823752  0.804104423    2.268053  2.554365e-02
## I(t^2)      -2.995557  0.007697489   -389.160315 1.023398e-156
```

- The GLS model gives pretty much the same coefficients as before, but the standard errors are larger.